Preorder traversal split structure se Binary Search Tree (BST) construct karne ka **Brute Force approach** kafi intuitive hai. Iska basic principle yeh hai ki preorder traversal ka sabse pehla element hamesha **Root Node** hota hai.

Is concept ko bilkul simple steps aur detailed logic ke sath samajhte hain.

Core Concept (The Property of BST)

BST ki sabse badi khubi yeh hoti hai:

- Root ke **Left Subtree** ke saare elements root se **chhote** honge.
- Root ke **Right Subtree** ke saare elements root se **bade** honge.

Aur Preorder ka order hota hai: **[Root → Left → Right]**

Brute Force Approach: One-by-One Insertion

Brute force tarike me hum kya karte hain? Hum preorder array ke pehle element ko Root bana dete hain. Uske baad, baaki bache saare elements par loop chalate hain aur ek-ek karke unhe BST me uski sahi jagah par **insert** karte hain.

Algorithm Steps:

1. Array ke pehle element (`preorder[0]`) ko uthao aur usse **Root Node** bana do.
2. Ab index `1` se lekar `n-1` tak ke elements par iterate karo.
3. Har element ke liye, Root se shuru karke niche tak jao (compare karte hue) aur sahi position par attach karo:

- Agar current element key se **chhota** hai, toh **left** me jao.
- Agar current element key se **bade** hai, toh **right** me jao.
- Jaise hi koi `null` spot mile, wahan naya node insert kar do.

Dry Run with an Example

Maan lete hain humare paas yeh preorder array hai: `[10, 5, 1, 7, 40, 50]`

- **Step 1:** Pehla element `10` hai. Ise humne **Root** bana diya.

```
10 (Root)
```

- **Step 2:** Agla element hai `5`.
- `5 < 10`, toh yeh `10` ke **left** me jayega. `10` ka left abhi khali (`null`) hai, toh wahan attach kar do.

```
   10
  /
 5
```

- **Step 3:** Agla element hai `1`.
- `1 < 10`, left me jao (ab hum `5` par hain).
- `1 < 5`, `5` ke left me jao. `5` ka left khali hai, toh wahan attach kar do.

```
     10
    /
   5
  /
 1
```

- **Step 4:** Agla element hai `7`.
- `7 < 10`, left me jao (hum `5` par hain).
- `7 > 5`, right me jao. `5` ka right khali hai, toh wahan attach kar do.

```
     10
    /
   5
  / \
 1   7
```

- **Step 5:** Agla element hai `40`.
- `40 > 10`, right me jao. `10` ka right khali hai, toh wahan attach kar do.

```
     10
    /  \
   5    40
  / \
 1   7
```

- **Step 6:** Aakhri element hai `50`.
- `50 > 10`, right me jao (`40` par aaye).
- `50 > 40`, `40` ke right me jao. Khali spot mila, attach kar do.

```
     10
    /  \
```

```
      5    40
     / \     \
    1   7     50
```

Aapka final BST taiyar hai!

Code Implementation (Java)

Yahan iska standard recursive insertion code hai jo Brute Force approach use karta hai:

```java
class Node {
    int data;
    Node left, right;

    Node(int val) {
        data = val;
        left = right = null;
    }
}

class BinarySearchTree {
    // Helper function ek single node ko BST me sahi jagah insert karne ke liye
    public Node insert(Node root, int data) {
        // Agar tree/subtree khali hai, toh naya node return karo
        if (root == null) {
            return new Node(data);
        }

        // Agar data chhota hai, toh left subtree me insert karo
        if (data < root.data) {
            root.left = insert(root.left, data);
        }
        // Agar data bada hai, toh right subtree me insert karo
        else if (data > root.data) {
            root.right = insert(root.right, data);
        }

        return root;
    }

    // Main function preorder se BST banane ke liye
    public Node bstFromPreorder(int[] preorder) {
        if (preorder == null || preorder.length == 0) return null;

        // Step 1: Pehle element ko root banao
        Node root = new Node(preorder[0]);

        // Step 2: Baaki saare elements ko ek-ek karke insert karo
        for (int i = 1; i < preorder.length; i++) {
```

```
            insert(root, preorder[i]);
        }

        return root;
    }
}
```

Complexity Analysis

Brute Force dekhne me simple hai, par yeh optimal nahi hai. Iski complexities samajhna zaroori hai:

1. Time Complexity:

- **Best / Average Case:** $O(N \log N)$ Agar tree **balanced** banta hai (jaise binary search tree banna chahiye), toh har element ko sahi jagah dhundhne me $\log N$ ka time lagta hai. N elements ke liye yeh $O(N \log N)$ ho jata hai.
- **Worst Case:** $O(N^2)$ Agar input array sorted ho (e.g., `[1, 2, 3, 4, 5]` ya `[5, 4, 3, 2, 1]`), toh tree ek **Skewed Tree** (line jaisa) ban jayega. Us case me har naye element ko insert karne ke liye poore tree ko traverse karna padega, jisse complexity $O(N^2)$ ho jayegi.

2. Space Complexity:

- $O(N)$ (Auxiliary Space) Worst case me recursion stack ki depth tree ki height ke barabar (N) ho sakti hai (Skewed tree ke liye). Average case me tree height $\log N$ hoti hai, toh stack space $O(\log N)$ hoga.

Is approach me koi bhi doubt ho toh pooch sakte ho! Agar isko optimal tareeqe ($O(N)$ range-bound solution) se samajhna hai, toh woh bhi bata dena.

Preorder traversal se BST construct karne ka **Optimal Approach** ise sirf $O(N)$ **time** me solve kar deta hai. Brute force ki tarah hume har baar root se shuru karke niche nahi jana padta, balki hum ek hi pass (single traversal) me pura tree design kar dete hain.

Ise samajhne ke liye pehle iski **Intuition** ko paka karte hain, phir algorithm aur code par aayenge.

The Core Intuition: "Range Bounding" (Laxman Rekha)

BST ki basic property ko hum sab jaante hain: `Left < Root < Right`.

Agar hum dhyan se sochein, toh tree ke har ek node ki apni ek **valid range (boundary)** hoti hai. Koi bhi naya node tabhi kisi jagah par fit ho sakta hai jab uski value us range ke andar fall kare.

Chalo ek simple real-world analogy lete hain:

> Socho aap ek line me khade logon ko unki height ke hisab se ek family tree me arrange kar rahe ho. Aap kisi naye member ko tabhi ek branch me bhejoge agar uski height us branch ki **Minimum** aur **Maximum** allowed limit ke andar hogi. Agar limit match nahi hoti, toh aap us branch ko closed maan kar agle side check karte ho.

Optimal approach me hum har ek node ke liye ek **Upper Bound (Maximum Limit)** set kar dete hain.

Hum sirf Upper Bound (Max Limit) se kaam kyun chala sakte hain?

Preorder ka nature hai: **[Root → Left → Right]** Hum hamesha left child ko pehle explore karte hain. Jab tak elements root se chhote mil rahe hain, woh left subtree me jate rahenge. Jaise hi koi element root se bada milega, hume samajh aa jayega ki left subtree ka limit khatam ho chuka hai, aur ab hume **Right Subtree** me move karna hai.

Isliye, hume bas har step par yeh track rakhna hai: "*Kya current element is subtree ki Upper Bound ke andar hai?*"

Detailed Step-by-Step Logic

Hum array me aage badhne ke liye ek **global index pointer** (`i = 0`) rakhenge aur ek recursive function chalayenge jisme hum ek `upper_bound` pass karenge.

Initial State me, Root node ke liye upper bound kya hoga? ∞ **(Infinity)**, kyunki root kuch bhi ho sakta hai.

Range Rule for Subtrees:

1. **Left Subtree me jaate waqt:** Upper bound badal kar **Current Node ki value** ho jayegi. (Kyunki left child hamesha current node se chhota hona chahiye).
2. **Right Subtree me jaate waqt:** Upper bound **vahi rahega** jo parent se mila tha. (Kyunki right child parent se bada ho sakta hai, par parent ke bhi jo upar tha usse chhota hona chahiye).

Dry Run with an Example

Wahi purana array lete hain: `[10, 5, 1, 7, 40, 50]` *Global Index `i = 0`*

- **Step 1: Function `solve(upper_bound = 1000)`** *(Maan lete hain infinity = 1000)*
- Array element `preorder[0] = 10`.
- Kya `10 < 1000`? Haan! Toh `10` ka node banao, aur index ko badha kar `i = 1` kar do.
- Ab yeh node apne **Left** aur **Right** ke liye calls lagayega.
- **Step 2: Left Call of 10 → `solve(upper_bound = 10)`**
- Current element `preorder[1] = 5`.
- Kya `5 < 10`? Haan! Toh `5` ka node banao aur use `10` ke left me attach karo. Index `i = 2`.
- Ab `5` apne Left aur Right ke liye calls lagayega.
- **Step 3: Left Call of 5 → `solve(upper_bound = 5)`**
- Current element `preorder[2] = 1`.
- Kya `1 < 5`? Haan! `1` ka node banao, use `5` ke left me attach karo. Index `i = 3`.

- **Step 4: Left Call of 1 → `solve(upper_bound = 1)`**
- Current element `preorder[3] = 7`.
- Kya `7 < 1`? **Nahi!** Toh yeh function `null` return karega (matlab `1` ka left khali rahega). Index `i` aage nahi badhega.
- **Step 5: Right Call of 1 → `solve(upper_bound = 5)`** *(1 ka right parent yaani 5 ke bound tak ja sakta hai)*
- Current element `preorder[3] = 7`.
- Kya `7 < 5`? **Nahi!** Yeh bhi `null` return karega (`1` ka right bhi khali).
- **Step 6: Right Call of 5 → `solve(upper_bound = 10)`** *(Hum wapas upar aaye, ab 5 ka right check ho raha hai)*
- Current element `preorder[3] = 7`.
- Kya `7 < 10`? **Haan!** Toh `7` ka node banao aur use `5` ke right me attach karo. Index `i = 4`.

Yeh process isi tarah chalti rahegi. Jaise hi `40` aayega, woh `10` ke left subtree ke saare bounds (`10`) ko tod dega, aur recursive functions return hote-hote seedhe `10` ke **Right Call** (`upper_bound = 1000`) me fit ho jayega.

Code Implementation (Java)

```
class Node {
    int data;
    Node left, right;

    Node(int val) {
        data = val;
        left = right = null;
    }
}

class Solution {
    // Array ko track karne ke liye ek global ya class-level pointer
    private int index = 0;

    public Node bstFromPreorder(int[] preorder) {
        // Shuruat me upper bound ko Integer.MAX_VALUE (Infinity) set karenge
        return constructBST(preorder, Integer.MAX_VALUE);
    }

    private Node constructBST(int[] preorder, int upperBound) {
        // Base Case 1: Agar saare elements process ho gaye hain
        // Base Case 2: Agar current element upper bound ko violate kar raha hai
        if (index >= preorder.length || preorder[index] > upperBound) {
            return null;
        }
```

```
        // Agar valid hai, toh node banao aur pointer aage badhao
        Node root = new Node(preorder[index++]);

        // Left subtree ke liye Upper Bound ab is 'root.data' ke barabar hoga
        root.left = constructBST(preorder, root.data);

        // Right subtree ke liye Upper Bound wahi rahega jo is node ko mila tha
        root.right = constructBST(preorder, upperBound);

        return root;
    }
}
```

Complexity Analysis (Why this is optimal)

1. Time Complexity: $O(N)$

Brute force me hum baar-baar jodte hue niche tak travel kar rahe the ($O(N^2)$). Lekin yahan, hum array ke har element par **sirf ek baar** ja rahe hain (`index++` sirf tabhi hota hai jab node banta hai). Har element par sirf ek $O(1)$ ka boundary check hota hai. Isliye total time complexity linear ho jaati hai, yaani $O(N)$.

2. Space Complexity: $O(N)$

- **Auxiliary Space:** Recursion stack ka space tree ki height (H) par depend karta hai.
- Worst case me (skewed tree) height N ho sakti hai, isliye space $O(N)$ hoga.
- Average/Best case (balanced tree) me height $\log N$ hoti hai, toh space $O(\log N)$ hoga.

Yahan optimal approach ($O(N)$ upper bound method) ka **C++ implementation** hai. Humne `index` ko pass-by-reference (`int& index`) rakha hai taaki poore recursive system me array ka current pointer update hota rahe.

C++ Implementation

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

// Node structure definition
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) {
        val = x;
        left = nullptr;
```

```
        right = nullptr;
    }
};

class Solution {
private:
    // Helper function jo recursive tree construct karega
    TreeNode* constructBST(vector<int>& preorder, int& index, int upperBound) {
        // Base Case 1: Agar array ke saare elements process ho chuke hain
        // Base Case 2: Agar current element upper bound se bada hai, toh yeh is
subtree ka part nahi hai
        if (index >= preorder.size() || preorder[index] > upperBound) {
            return nullptr;
        }

        // Sahi position milne par naya node banao aur index ko aage badhao
        TreeNode* root = new TreeNode(preorder[index++]);

        // Left child ke liye upper bound 'root->val' ban jayega
        root->left = constructBST(preorder, index, root->val);

        // Right child ke liye upper bound wahi rahega jo parent ka tha
        root->right = constructBST(preorder, index, upperBound);

        return root;
    }

public:
    TreeNode* bstFromPreorder(vector<int>& preorder) {
        int index = 0;
        // Shuruat me upper bound ko INT_MAX (Infinity) pass karenge
        return constructBST(preorder, index, INT_MAX);
    }
};

// Helper function to print tree in inorder (sorted order verification ke liye)
void printInorder(TreeNode* root) {
    if (root == nullptr) return;
    printInorder(root->left);
    cout << root->val << " ";
    printInorder(root->right);
}

int main() {
    Solution solver;
    vector<int> preorder = {10, 5, 1, 7, 40, 50};

    TreeNode* root = solver.bstFromPreorder(preorder);

    cout << "Inorder traversal of created BST (Should be sorted): " << endl;
    printInorder(root);
    cout << endl;

    return 0;
```

```
}
```

Key Highlights of C++ Code:

1. `int& index` **(Pass by Reference):** Yeh sabse important part hai. C++ me agar aap simple integer pass karoge toh woh copy ho jayega. Reference use karne se jab `left subtree` apna processing karke `index` ko badhayega, toh updated `index` seedhe `right subtree` ko milega.
2. `INT_MAX`**:** Yeh `<climits>` header se aata hai, jo infinity (∞) ka kaam karta hai.
3. **Memory Optimization:** Isme hum vector ki koi copy nahi bana rahe hain, bas ek single reference array use ho raha hai, jo space optimize karta hai.

Postorder traversal se Binary Search Tree (BST) construct karne ka **Brute Force approach** kaafi hadd tak preorder jaisa hi hota hai, bas isme hum peeche se shuru karte hain.

Postorder ka standard order hota hai: **[Left** $\rightarrow$ **Right** $\rightarrow$ **Root]** Iska matlab yeh hai ki pure array ka jo **sabse aakhri element** hoga, woh hamesha pure tree ka **Main Root** hoga.

Core Logic & Intuition

Hum jante hain ki BST me root se chhote elements left me jaate hain aur bade elements right me.

Postorder array me elements ko right-to-left scan karne par hume sabse pehle **Root** milta hai, uske just pehle **Right Subtree** ke elements hote hain, aur shuruat me **Left Subtree** ke elements hote hain.

Hum brute force me kya karenge:

1. Array ke **aakhri element** ko utha kar use **Root Node** bana denge.
2. Uske baad, bache hue elements par peeche se aage ki taraf (yaani index `n-2` se `0` tak) ek loop chalayenge.
3. Har ek element ko utha kar BST ke property ke hisab se root se compare karte hue niche tak jayenge aur uski sahi jagah par **insert** kar denge.

Detailed Dry Run with an Example

Maan lete hain humare paas yeh postorder array hai: `[1, 7, 5, 50, 40, 10]` *(Yahan* `n = 6`*, toh hum reverse loop chalayenge index 5 se 0 tak)*

- **Step 1 (Index 5):** Aakhri element `10` hai. Ise humne **Root** bana diya.

```
10 (Root)
```

- **Step 2 (Index 4):** Agla element peeche se `40` hai.

- Compare karo: `40 > 10`, toh yeh `10` ke **right** me jayega. `10` ka right abhi khali (`nullptr`) hai, toh wahan attach kar do.

```
    10
      \
       40
```

- **Step 3 (Index 3):** Agla element `50` hai.
- `50 > 10`, right me jao (ab hum `40` par hain).
- `50 > 40`, `40` ke right me jao. Khali spot mila, attach kar do.

```
    10
      \
       40
         \
          50
```

- **Step 4 (Index 2):** Agla element `5` hai.
- `5 < 10`, `10` ke **left** me jao. Left khali hai, toh attach kar do.

```
      10
     /  \
    5    40
           \
            50
```

- **Step 5 (Index 1):** Agla element `7` hai.
- `7 < 10`, left me jao (hum `5` par hain).
- `7 > 5`, `5` ke **right** me jao. Spot khali hai, attach kar do.

```
      10
     /  \
    5    40
     \     \
      7     50
```

- **Step 6 (Index 0):** Aakhri bacha element `1` hai.
- `1 < 10`, left me jao (`5` par aaye).
- `1 < 5`, `5` ke **left** me jao. Spot khali hai, attach kar do.

Humara final BST kuch aisa dikhega:

```
        10
       /  \
      5    40
     / \     \
    1   7     50
```

C++ Code Implementation (Brute Force)

Hum standard `insert` function ka use karenge jo naye node ko BST ke rules ke mutabik top-to-bottom traverse karke attach karta hai.

```cpp
#include <iostream>
#include <vector>

using namespace std;

// Node structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Helper function ek naye node ko BST me sahi jagah insert karne ke liye
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        // Agar current spot khali hai, toh naya node bana kar return karo
        if (root == nullptr) {
            return new TreeNode(val);
        }

        // Agar value root se chhoti hai, toh left me insert karo
        if (val < root->val) {
            root->left = insertIntoBST(root->left, val);
        }
        // Agar value root se badi hai, toh right me insert karo
        else if (val > root->val) {
            root->right = insertIntoBST(root->right, val);
        }

        return root;
    }

    TreeNode* bstFromPostorder(vector<int>& postorder) {
```

```
        if (postorder.empty()) return nullptr;

        int n = postorder.size();
        // Step 1: Sabse aakhri element ko root banao
        TreeNode* root = new TreeNode(postorder[n - 1]);

        // Step 2: Reverse loop chalao (n-2 se lekar 0 tak) aur ek-ek karke insert
karo
        for (int i = n - 2; i >= 0; i--) {
            insertIntoBST(root, postorder[i]);
        }

        return root;
    }
};

void printInorder(TreeNode* root) {
    if (!root) return;
    printInorder(root->left);
    cout << root->val << " ";
    printInorder(root->right);
}

int main() {
    Solution solver;
    vector<int> postorder = {1, 7, 5, 50, 40, 10};

    TreeNode* root = solver.bstFromPostorder(postorder);

    cout << "Inorder traversal of constructed BST: ";
    printInorder(root); // Output sorted aana chahiye: 1 5 7 10 40 50
    cout << endl;

    return 0;
}
```

Complexity Analysis

Brute Force hone ki wajah se iski complexities preorder brute-force ke jaisi hi hoti hain:

1. Time Complexity

- **Worst Case:** $O(N^2)$ Agar input array already sorted order me ho (jaise `[5, 4, 3, 2, 1]`), toh tree ek extreme line (Skewed Tree) jaisa banega. Har naye node ko insert karne ke liye hume poore tree ko upar se niche tak scan karna padega, jisse total operations $N + (N-1) + (N-2) + ... + 1 = O(N^2)$ ho jayenge.
- **Average / Best Case:** $O(N \log N)$ Agar tree complete ya balanced banta hai, toh har node ko insert karne ki height search complexity $\log N$ hoti hai. N elements ke liye total time $O(N \log N)$ lagega.

2. Space Complexity: $O(N)$

- Yeh recursion stack space hai. Worst case me skewed tree banne par stack ki depth N tak ja sakti hai, isliye $O(N)$ space complexity hoti hai. Balanced tree me yeh $O(\log N)$ rahegi.

Postorder se BST banane ka **Optimal Approach** dhyan se samajhte hain. Jab tumse koi interviewer pooche ki "*Tumne yeh kaise socha?*" ya "*Iska thought process kya hai?*", toh tum unhe yeh step-by-step intuition de sakte ho.

Is approach se hum poore problem ko $O(N)$ time me bina kisi extra data structure ke solve kar dete hain.

---

1. Thought Process (Kaise Socha?)

Chalo step-by-step dimaag lagate hain:

- **Observation 1 (The Postorder Nature):** Postorder hota hai `[Left -> Right -> Root]`. Iska matlab agar hum array me **peeche se aage (Right to Left)** travel karein, toh sabse pehle hume **Root** milega. Root ke just pehle hume **Right Subtree** ke elements milenge, aur uske pehle **Left Subtree** ke elements milenge.

> **Reverse Postorder Path:** `Root -> Right -> Left`

- **Observation 2 (The BST Property):** BST me har node ki apni ek **Laxman Rekha (Range/Boundary)** hoti hai.
- Agar hum kisi node ke **Right** me ja rahe hain, toh us naye node ki value current root se **badi** honi chahiye, par uske upar jo ancestors hain unke bounds ke andar honi chahiye.
- Agar hum kisi node ke **Left** me ja rahe hain, toh value current root se **chhoti** honi chahiye.
- **The "Aha!" Moment (Connecting both):** Hum preorder wale optimal solution me **Upper Bound** use kar rahe the kyunki path `Root -> Left -> Right` tha. Lekin postorder me hum peeche se chal rahe hain, toh humara path hai `Root -> Right -> Left`. Yahan jab hum right subtree explore karte hain, toh values badhti hain. Isliye hume track rakhna padega ek **Lower Bound (Minimum Allowed Value)** ka! Jab tak hume peeche aane par aisi values mil rahi hain jo current root se **badi** hain, woh uske right subtree ka hissa banti jayengi. Jaise hi koi aisi value milegi jo lower bound ko violate karegi (yaani root se chhoti hogi), hum samajh jayenge ki Right Subtree ka quota khatam, ab yeh element **Left Subtree** me jayega.

---

2. Range Rules for Postorder (Lower Bound Method)

Hum ek global index pointer rakhenge jo array ke aakhri element (`n-1`) se shuru hoga aur `0` tak jayega. Hum har recursive call me ek `lower_bound` pass karenge:

1. **Initial State:** Main Root ke liye lower bound kya hoga? $-\infty$ **(Minus Infinity)**, kyunki root kuch bhi ho sakta hai.
2. **Right Subtree me jaate waqt:** Lower bound badal kar **Current Node ki value** ho jayegi. (Kyunki right child hamesha current node se bada hona chahiye).
3. **Left Subtree me jaate waqt:** Lower bound **vahi rahega** jo use parent se mila tha.

3. Detailed Dry Run with an Example

Maan lete hain humare paas array hai: `[1, 7, 5, 50, 40, 10]` Hum shuru karenge peeche se, yaani **`index = 5`** (value = 10).

- **Step 1: `solve(lower_bound = -∞)**`
- `postorder[5] = 10`. Kya `10 > -∞`? Haan!
- `10` ka node ban gaya. `index` ghat kar `4` ho gaya.
- Ab order `Root -> Right -> Left` hai, toh pehle **Right Call** lagegi, phir **Left Call**.
- **Step 2: Right Call of 10 → `solve(lower_bound = 10)**`
- Current element `postorder[4] = 40`.
- Kya `40 > 10`? Haan! `40` ka node banao aur `10` ke right me jodo. `index` ab `3` ho gaya.
- **Step 3: Right Call of 40 → `solve(lower_bound = 40)**`
- Current element `postorder[3] = 50`.
- Kya `50 > 40`? Haan! `50` ka node banao aur `40` ke right me jodo. `index` ab `2` ho gaya.
- **Step 4: Right Call of 50 → `solve(lower_bound = 50)**`
- Current element `postorder[2] = 5`.
- Kya `5 > 50`? **Nahi!** Toh yeh call `nullptr` return karegi. `index` vahi rukega (`2`).
- **Step 5: Left Call of 50 → `solve(lower_bound = 40)**` *(Parent 40 ka lower bound mila)*
- Current element `postorder[2] = 5`. Kya `5 > 40`? **Nahi!** `nullptr` return.
- **Step 6: Left Call of 40 → `solve(lower_bound = 10)**` *(Grandparent 10 ka lower bound mila)*
- Current element `postorder[2] = 5`. Kya `5 > 10`? **Nahi!** `nullptr` return.
- **Step 7: Left Call of 10 → `solve(lower_bound = -∞)**` *(Hum wapas main root par aa gaye)*
- Current element `postorder[2] = 5`.
- Kya `5 > -∞`? **Haan!** Toh `5` ka node banao aur use `10` ke left me attach karo. `index` ghat kar `1` ho gaya.

Yeh process isi tarah aage chalti rahegi jab tak `index < 0` nahi ho jata.

---

4. C++ Code (Optimal Approach)

```cpp
#include <iostream>
#include <vector>
#include <climits>
```

```cpp
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    // Helper function jo reverse postorder traversal karega (Root -> Right -> Left)
    TreeNode* constructBST(vector<int>& postorder, int& index, int lowerBound) {
        // Base Case 1: Agar saare elements process ho gaye hain
        // Base Case 2: Agar current element lower bound se chhota hai, toh yeh is subtree ka part nahi hai
        if (index < 0 || postorder[index] < lowerBound) {
            return nullptr;
        }

        // Sahi position milne par node banao aur index ko peeche shift karo (decrement)
        TreeNode* root = new TreeNode(postorder[index--]);

        // Pehle RIGHT call lagegi kyunki hum peeche se aa rahe hain (Postorder: Left, Right, Root)
        // Right child ke liye Lower Bound ab 'root->val' ban jayega
        root->right = constructBST(postorder, index, root->val);

        // Phir LEFT call lagegi
        // Left child ke liye Lower Bound wahi rahega jo parent ka tha
        root->left = constructBST(postorder, index, lowerBound);

        return root;
    }

public:
    TreeNode* bstFromPostorder(vector<int>& postorder) {
        int index = postorder.size() - 1; // Last index se shuru karenge
        // Shuruat me lower bound ko INT_MIN (-Infinity) rakhenge
        return constructBST(postorder, index, INT_MIN);
    }
};
```

---

5. Complexity Breakdown

- **Time Complexity:** $O(N)$ Hum array ke har element ko peeche se sirf ek baar visit kar rahe hain (`index--` tabhi hota hai jab node successfully banta hai). Har check sirf $O(1)$ ka constant comparison

hai. Isliye yeh linear time me solve ho jata hai.
- **Space Complexity:** $O(N)$ Recursion stack ka space tree ki height ke barabar hota hai. Worst case (skewed tree) me $O(N)$ aur average case (balanced tree) me $O(\log N)$ space lagega.

Acha, pehle **Brute Force approach** ko bilkul deep me jakar samajhte hain ki bina kisi stack ya advanced dimag lagaye, ek aam insaan is problem ko kaise sochega.

---

1. Brute Force ki Core Intuition

Agar tumhe koi ek preorder array de de, toh valid BST check karne ka sabse basic tarika kya hai? Har ek node ke liye hume check karna padega ki kya woh BST ke rules ko follow kar raha hai.

BST ka rule kya hai?

- `Left Subtree` ke saare elements root se **chhote** hone chahiye.
- `Right Subtree` ke saare elements root se **bade** hone chahiye.

Aur Preorder ka structural behavior kya hota hai? **[Root | Left Subtree | Right Subtree]**

Agar koi array valid preorder hai, toh root ke just baad pehle saare chhote elements (Left Subtree) aane chahiye, aur jaise hi pehla bada element (Right Subtree ka start) mile, uske baad **saare ke saare elements root se bade hi hone chahiye**. Agar right subtree wale area me koi bhi element root se chhota mil gaya, toh samajho game over, valid BST possible nahi hai!

---

2. Detailed Algorithm Steps

Hum har ek element ko bari-bari se `Root` maan kar pure array par check lagayenge:

1. Ek outer loop chalayenge jo array ke har element `i` par jayega. Yeh `preorder[i]` humara **Current Root** hai.
2. Ab is root ke just aage se (`i + 1` se) hum check karna shuru karenge:

- **Phase 1 (Find Left Subtree):** Hum tab tak aage badhenge jab tak elements `preorder[i]` se **chhote** hain. Yeh saare elements left subtree ka hissa hain.
- **Phase 2 (Check Right Subtree):** Jaise hi hume pehla aisa element mile jo `preorder[i]` se **bada** hai, hum samajh jayenge ki yahan se Right Subtree shuru ho gaya hai. Ab yahan se lekar array ke end tak, hume check karna hai ki **koi bhi element root se chhota nahi hona chahiye**.

3. Agar right subtree wale area me ek bhi element root se chhota mil gaya, toh wahi se `false` return kar do.
4. Agar pure array ke har node ke liye yeh condition true baithti hai, toh end me `true` return karo.

---

3. Detailed Dry Run with an Example

Chalo ek **Invalid Example** lekar dekhte hain taaki samajh aaye ki brute force ise pakadta kaise hai. Array: `[4, 2, 5, 1]`

- **Loop `i = 0` (Current Root = 4):**

- Hum `i+1` (yaani index 1) se scan shuru karenge.
- *Index 1 (Value = 2):* `2 < 4`. Yeh left subtree me hai. Aage badho.
- *Index 2 (Value = 5):* `5 > 4`. Yahan se **Right Subtree** shuru ho gaya! Ab humara rule hai ki aage sab kuch `4` se bada hona chahiye.
- *Index 3 (Value = 1):* Humne check kiya `1 < 4`. **Violation!** Right subtree area me `4` se chhota element `1` mil gaya.
- Loop aage check hi nahi karega, seedhe return `false`.

---

4. C++ Code (Brute Force Approach)

```cpp
#include <iostream>
#include <vector>

using namespace std;

class Solution {
public:
    bool verifyPreorderBruteForce(vector<int>& preorder) {
        int n = preorder.size();

        // Har ek element ko individual root maan kar check karenge
        for (int i = 0; i < n; i++) {
            int root = preorder[i];

            // Step 1: Left subtree ke saare elements ko skip karo (jo root se
chhote hain)
            int j = i + 1;
            while (j < n && preorder[j] < root) {
                j++;
            }

            // 'j' ab pehle bade element par khada hai (Right subtree ki shuruat)
            // Step 2: Ab yahan se lekar end tak koi bhi element root se chhota
nahi hona chahiye
            for (int k = j; k < n; k++) {
                if (preorder[k] < root) {
                    return false; // Agar right subtree me chhota element mila,
toh invalid
                }
            }
        }

        return true; // Agar sabhi nodes ke liye rule pass ho gaya
    }
};

int main() {
    Solution solver;
    vector<int> invalid_case = {4, 2, 5, 1};
    vector<int> valid_case = {10, 5, 1, 7, 40, 50};
```

```
    cout << "Invalid Case: " << (solver.verifyPreorderBruteForce(invalid_case) ?
"True" : "False") << endl;
    cout << "Valid Case: " << (solver.verifyPreorderBruteForce(valid_case) ?
"True" : "False") << endl;

    return 0;
}
```

---

5. Complexity Analysis (Brute Force Kyun Bura Hai?)

Time Complexity: $O(N^2)$

Hum ek outer loop chala rahe hain N tak, aur uske andar har baar bache hue array ko scan kar rahe hain (Worst case me lagbhag N elements scan hote hain).

- **Worst Case Scenario:** Agar array ordered ho jaise `[1, 2, 3, 4, 5]`, toh har baar left subtree milega hi nahi, direct right subtree check hoga aur poora array scan hoga. Total comparisons $\approx N + (N-1) + (N-2) + ... \rightarrow O(N^2)$.

Space Complexity: $O(1)$

Hum koi extra stack, queue ya recursion use nahi kar rahe hain. Bas do pointers (`j` aur `k`) se iteration ho rahi hai, isliye space complexity absolute constant hai.

Brute force ka poora maza isme hai ki yeh kafi simple hai, par $O(N^2)$ time interview me accept nahi hoga. Iske baad hi woh stack wala $O(N)$ optimal dimag me aata hai jo humne pehle discuss kiya tha. Kya ab dono approach ke beech ka farq bilkul clear hai?

Brute Force me dikkat kya thi? Hum har ek element par khade hokar baar-baar aage ke pure array ko scan kar rahe the ($O(N^2)$).

Optimal solution tak pahunchne ke liye hume khud se ek sawal poochna padega: **"Kya hum koi aisi information maintain kar sakte hain jisse hume baar-baar aage ka array scan na karna pade, aur ek hi baar me kaam ho jaye?"**

Chalo iska poora thought process aur intuition bilkul dhyan se samajhte hain ki ek expert developer is $O(N)$ approach ko kaise sochta hai.

---

1. Breakdown the Thought Process (Kaise Socha?)

Step A: Preorder ke patterns ko dhyan se observe karo

Preorder ka order hota hai: `[Root -> Left Subtree -> Right Subtree]`

Maan lo humare paas ek valid sequence hai: `[10, 5, 2, 7, 12]`

- `10` se shuru kiya (Root).
- `5` aaya: Yeh `10` se chhota hai, toh yeh left subtree me gaya.
- `2` aaya: Yeh `5` se chhota hai, toh yeh `5` ke bhi left me gaya.

Yahan tak dekho, elements **decrease** hote ja rahe hain (`10 -> 5 -> 2`). Jab tak elements chhote hote ja rahe hain, iska matlab hum tree me niche aur niche **Left Subtree** ki taraf badhte ja rahe hain.

Step B: The Turning Point (Bada Element Aana)

Ab suddenly `2` ke baad array me aaya `7`. `7` toh `2` se bada hai! Iska matlab ab Left Subtree ka downward path toot gaya. Hum ab **Right Subtree** me enter kar rahe hain.

Lekin yeh `7` kiska right child hai?

- Kya yeh `2` ka right child hai? Haan, ho sakta hai kyunki `7 > 2`.
- Kya yeh `5` ka right child hai? Haan, ho sakta hai kyunki `7 > 5`.
- Kya yeh `10` ka right child hai? Nahi, kyunki agar `10` ka right hota toh `10` se bada hona chahiye tha, par `7 < 10` hai.

Yahan se hume samajh aaya ki `7` actually `5` ka right child hai. Jaise hi humne yeh decide kiya ki hum `5` ke right subtree me aa chuke hain, iska matlab humne `5` **ke pure left subtree (yaani `2`) ko hamesha ke liye peeche chhod diya hai.**

Step C: The Ultimate BST Rule (The Threshold)

BST ka sabse kadwa niyam: **Ek baar jab aap kisi node ke right subtree me chale gaye, toh aage aane wala koi bhi element us node se chhota nahi ho sakta.**

Kyunki hum `5` ke right me aa chuke hain, toh ab aage array me koi bhi element `5` se chhota nahi aana chahiye! `5` humara ek **Lower Bound (Threshold)** ban gaya hai.

---

2. Is Soch Ko Stack Me Kaise Badla? (The Mechanism)

Baar-baar peeche ke nodes ko check karne ke liye hume ek aisi chiz chahiye jo elements ko reverse order me track kare. Iske liye **Stack** sabse best hai!

1. **Decreasing Order me Stack:** Jab tak chhote elements mil rahe hain, unhe stack me push karte jao (kyunki woh left subtree bana rahe hain).
2. **Right Turn Detection:** Jaise hi koi bada element mile, stack se tab tak elements ko nikalte (`pop` karte) jao jab tak stack ka top chhota hai.
3. **Threshold Update:** Jo aakhri element stack se pop hoga, wahi humara sabse immediate parent (`root_value`) banega jiske right me hum ja rahe hain. Uske baad hum upna threshold badha denge.

---

3. Real Example Se Poora Khel Samajhte Hain

Chalo ek **Invalid Array** lete hain: `[4, 2, 5, 1]` *Shuruat me* `root_value = -∞` *(Koi restriction nahi hai)*

- **Element = 4:**

- Kya `4 < -∞`? Nahi. Rule pass.
- Stack khali hai, `4` ko push karo. `Stack = [4]`
- **Element = 2:**
- Kya `2 < -∞`? Nahi. Rule pass.
- `2` chhota hai `4` se (stack top), matlab hum abhi bhi left subtree me hain.
- `2` ko push karo. `Stack = [4, 2]`
- **Element = 5:**
- Kya `5 < -∞`? Nahi. Rule pass.
- Ab `5` bada hai stack ke top (`2`) se! Matlab right turn aaya.
- **Pop Loop:**
- `5 > 2` → `2` ko pop karo, `root_value = 2`. `Stack = [4]`
- `5 > 4` → `4` ko pop karo, `root_value = 4`. `Stack = []`
- Ab `5` ko stack me push karo. `Stack = [5]`.
- *(Ab humara active rule set ho chuka hai: `root_value = 4`. Yani aage koi bhi element 4 se chhota nahi ho sakta kyunki hum 4 ke right subtree me hain).*
- **Element = 1:**
- Check karo: Kya `1 < root_value (4)`? **Haan!**
- **Catch:** Kyunki hum `4` ke right me `5` ko bitha chuke hain, toh `4` ke khandan me ab `4` se chhota element `1` aa hi nahi sakta!
- Yeh pakda gaya chor! Turant return kar do `false`.

---

4. Final C++ Optimal Code

Yahi sochte hue humne yeh neat aur clean code design kiya:

```cpp
#include <iostream>
#include <vector>
#include <stack>
#include <climits>

using namespace std;

class Solution {
public:
    bool verifyPreorderOptimal(vector<int>& preorder) {
        stack<int> s;
```

```
        int root_value = INT_MIN; // Yeh humari restriction line (threshold) hai

        for (int element : preorder) {
            // Agar koi element restriction ko todta hai
            if (element < root_value) {
                return false;
            }

            // Agar naya element bada hai, yaani right side ja rahe hain
            // Pop karke tab tak peeche jao jab tak sahi parent nahi mil jata
            while (!s.empty() && element > s.top()) {
                root_value = s.top(); // Sabse last popped element hi root banega
                s.pop();
            }

            // Element ko stack me daal do
            s.push(element);
        }

        return true;
    }
};
```

Summary of Thought Process

Interviewer ke samne bas yeh 3 points bolna:

1. "Mene observe kiya ki preorder me jab tak elements ghat rahe hain, hum **Left** ja rahe hain."
2. "Jaise hi element badha, hum **Right** mude. Right mudte hi peeche chhoote hue elements se chhota element aage nahi aa sakta (BST Rule)."
3. "Peeche chhoote hue elements ka track rakhne ke liye mene **Monotonic Decreasing Stack** ka use kiya, jisse har element maximum ek baar push aur ek baar pop hota hai, aur time complexity $O(N^2)$ se ghat kar $O(N)$ ho jaati hai."

Binary Tree ya Binary Search Tree me **Lowest Common Ancestor (LCA)** dhoondhne ka problem ekdam classic hai. Interviewer jab aapse iska **Brute Force approach** poochta hai, toh woh aapki basic tree traversal aur array/vector manipulation ki skills check kar raha hota hai.

Chalo isko bilkul aam bhasha me, core intuition aur detailed dry run ke sath samajhte hain.

LCA Ka Matlab Kya Hai? (The Core Definition)

Maan lo aapke paas ek tree hai aur do nodes hain: `p` aur `q`.

- **Ancestor:** Kisi node ke upar wale saare parents (including itself) uske ancestors hote hain.
- **Common Ancestor:** Aise nodes jo `p` ke bhi upar aate hain aur `q` ke bhi upar aate hain.
- **Lowest Common Ancestor (LCA):** Woh sabse *niche* wala common node jo dono ke raste (path) me common ho.

Brute Force Ka Thought Process (Kaise Sochein?)

Brute force tarike me hum bilkul seedha aur simple dimaag lagate hain:

> "Agar mujhe Root se lekar Node `p` tak ka poora rasta (path) pata chal jaye, aur Root se lekar Node `q` tak ka bhi poora rasta pata chal jaye... toh main dono raston ko shuruat se compare kar sakta hoon. Jahan tak dono raste bilkul same chalenge, woh aakhri common point hi mera LCA hoga!"

Algorithm Steps:

1. Ek function banao jo Root se kisi target node tak ka **Path (Rasta)** dhoondh kar ek list/vector me store kare.
2. Is function ka use karke do alag alag lists banao:

- `path1`: Root se lekar Node `p` tak ka rasta.
- `path2`: Root se lekar Node `q` tak ka rasta.

3. Agar `p` ya `q` me se koi tree me present hi nahi hai, toh LCA possible nahi hai (return `nullptr`).
4. Ab dono lists (`path1` aur `path2`) par ek sath shuruat (index `0`) se iterate karo.
5. Jab tak dono lists ke elements match kar rahe hain, aage badhte jao.
6. **Jahan par dono lists ke elements mismatch ho jayein**, uske just pehle wala element hi aapka **Lowest Common Ancestor** hoga.

Detailed Dry Run with an Example

Maan lete hain humare paas yeh Binary Tree hai:

```
        3
      / \
     5   1
    / \ / \
   6  2 0  8
     / \
    7   4
```

Hume LCA dhoondhna hai `p = 7` aur `q = 4` ka.

Step 1: Find Path for `p = 7`

Hum root `3` se shuru karke `7` tak ka rasta nikalenge:

- `3 -> 5 -> 2 -> 7`
- `path1 = [3, 5, 2, 7]`

Step 2: Find Path for `q = 4`

Hum root `3` se shuru karke `4` tak ka rasta nikalenge:

- `3 -> 5 -> 2 -> 4`
- `path2 = [3, 5, 2, 4]`

Step 3: Compare both paths side-by-side

Ab hum dono arrays ko index 0 se match karna shuru karenge:

Index	`path1` element	`path2` element	Status	Last Common
0	3	3	Match!	3
1	5	5	Match!	5
2	2	2	Match!	2 (Lowest Tak Pahunch Gaye)
3	7	4	**Mismatch!**	Loop Breaks

Dono raste index 2 tak bilkul sath chal rahe the. Index 2 par jo node hai (yaani 2), wahi in dono ka sabse nichla aur aakhri common point hai. Toh humara **LCA = 2**.

C++ Code Implementation (Brute Force)

```cpp
#include <iostream>
#include <vector>

using namespace std;

// Node structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    // Helper function jo Root se Target node tak ka path find karta hai
    bool findPath(TreeNode* root, TreeNode* target, vector<TreeNode*>& path) {
        // Base Case: Agar node null hai toh path nahi mil sakta
        if (root == nullptr) return false;

        // Current node ko path me daalo
        path.push_back(root);

        // Agar target node mil gaya, toh true return karo
        if (root == target) return true;

        // Left ya Right subtree me target ko dhoondho
        if (findPath(root->left, target, path) || findPath(root->right, target,
path)) {
            return true;
```

```
        }

        // Agar dono side target nahi mila, toh is node ko path se nikal do
(Backtrack)
        path.pop_back();
        return false;
    }

public:
    TreeNode* lowestCommonAncestorBruteForce(TreeNode* root, TreeNode* p,
TreeNode* q) {
        vector<TreeNode*> path1, path2;

        // Step 1 & 2: Dono nodes ke liye paths nikal lo
        if (!findPath(root, p, path1) || !findPath(root, q, path2)) {
            return nullptr; // Agar koi ek node bhi nahi mila
        }

        // Step 3: Dono paths ko compare karo
        int i = 0;
        while (i < path1.size() && i < path2.size()) {
            if (path1[i] != path2[i]) {
                break; // Jahan mismatch hua, loop rok do
            }
            i++;
        }

        // Mismatch se theek pehle wala node (i-1) hi LCA hai
        return path1[i - 1];
    }
};
```

Complexity Analysis (Brute Force Kyun Optimal Nahi Hai?)

1. Time Complexity: $O(N)$

- `findPath` function `p` ke liye poore tree ko traverse kar sakta hai $\rightarrow O(N)$
- `findPath` function `q` ke liye bhi poore tree ko traverse kar sakta hai $\rightarrow O(N)$
- Paths ko compare karne me maximum $O(N)$ ka time lag sakta hai.
- **Total Time:** $O(N) + $O(N) + $O(N) = O(N)$.

> *Note: Bhale hi iski time complexity linear hai, par hum pure tree ko **multiple passes** (do-teen baar) me traverse kar rahe hain.*

2. Space Complexity: $O(N)$

- **Extra Space:** Humne do vectors (`path1` aur `path2`) banaye hain nodes ke raste ko store karne ke liye. Skewed tree ke case me in vectors ka size N tak ja sakta hai, isliye extra space complexity $O(N)$ lagti hai.

- **Recursion Stack Space:** $O(H)$ jahan H tree ki height hai.

Interviewer is brute force ke baad aapse hamesha kahega: **"Kya tum bina yeh extra paths (`vector`) store kiye, single traversal ($O(1)$ extra space) me ise solve kar sakte ho?"** (Jo iska optimal recursive approach banta hai).

LCA (Lowest Common Ancestor) dhoondhne ka **Optimal Approach** ise sirf **ek single traversal (One-Pass) me aur bina kisi extra vector/array storage ke ($O(1)$ Extra Space)** solve kar deta hai.

Iska poora thought process, intuition aur code bilkul detail me samajhte hain.

---

1. Optimal Thought Process (Kaise Sochein?)

Brute force me hume upar se niche tak ka rasta track karna pad raha tha kyunki hum upar khade hokar soch rahe the. Optimal approach me hum **Bottom-Up (Niche se upar)** sochenge using **Recursion**.

Socho tum `Root` par khade ho aur tumne apne dono bacho (`Left` aur `Right`) ko ek kaam par bheja: *"Jao aur jaakar pata karo ki kya tumhare subtrees me `p` ya `q` chhup kar baithe hain?"*

Ab niche se tumhare paas 3 tarah ke answers aa sakte hain:

1. **Node mil gaya:** Agar kisi side use `p` ya `q` mil jata hai, toh woh us node ka address wapas upar bhej dega.
2. **Kuch nahi mila:** Agar kisi side na `p` mila na `q`, toh woh chupchaap `null` return kar dega.

The Deciding Moment (LCA ki Property)

Jab tum kisi node par khade ho aur tumhare dono bacho se answers wapas aate hain, toh tum teen conditions check karoge:

- **Condition 1 (Dono side se answer aaya):** Agar tumhare Left subtree se bhi koi valid node return hua aur Right subtree se bhi koi valid node return hua... iska matlab **ek target tumhare left me hai aur dusra target tumhare right me hai**.

> Agar tum aisi position par ho jahan ek target left me hai aur ek right me, toh **tum khud hi un dono ke sabse immediate common parent (LCA) ho!** Tum apna address upar bhej doge.

- **Condition 2 (Sirf ek side se answer aaya):** Agar ek side se `null` aaya aur dusri side se koi valid node aaya... iska matlab dono ke dono targets usi ek single subtree ke andar maujood hain. Toh jo valid node niche se mila hai, wahi aage upar pass ho jayega.
- **Condition 3 (Dono side se `null` aaya):** Iska matlab is pure area me na `p` hai na `q`. Chupchaap `null` upar bhej do.

---

2. Recursive Rules (Base Cases)

Jab hum kisi node par traverse karte hue pahunchenge:

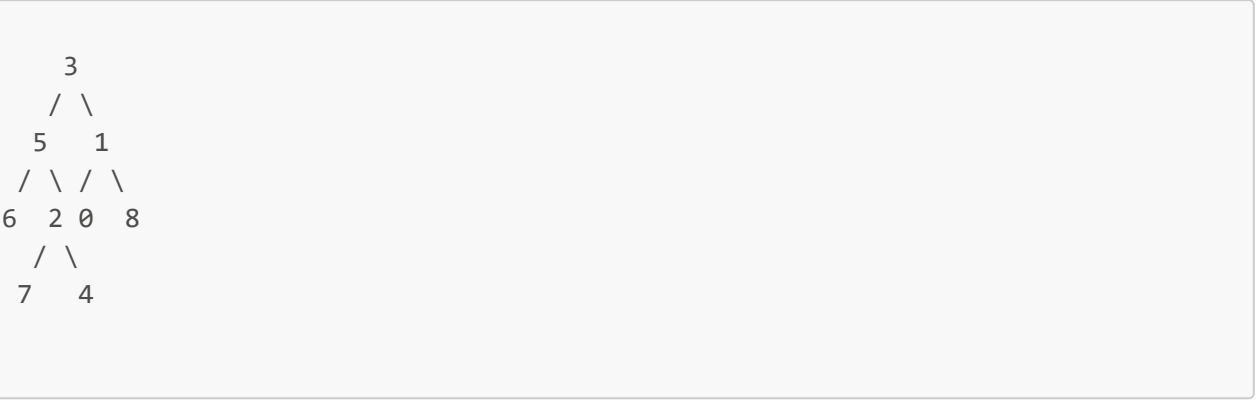
1. Agar current node `null` hai, toh `null` return karo.
2. Agar current node khud `p` ya `q` me se koi ek hai, toh aage niche check karne ki zaroorat hi nahi hai! Kyunki agar ek target khud root ban gaya, toh wahi potential LCA ho sakta hai (ya uske upar wala koi

hoga). Toh hum usi **current node** ko upar return kar dete hain.

---

3. Detailed Dry Run with an Example

Wahi purana tree lete hain:



Hume LCA dhoondhna hai `p = 7` aur `q = 4` ka.

- **Step 1:** Main root `3` se call shuru hui. `3` ne apne left (`5`) aur right (`1`) ko bola dhoondhne ko.
- **Step 2: Left side processing (`3 -> 5 -> 6`)**
- `5` ne apne left `6` par call lagayi. `6` ke left/right dono `null` hain. `6` khud `p` ya `q` nahi hai. Toh `6` ne return kiya **`null`** kisko? `5` ko.
- **Step 3: Right side of 5 processing (`5 -> 2 -> 7`)**
- `5` ne apne right `2` ko bola. `2` ne apne left `7` par call lagayi.
- `7` khud humara target `p` hai! Base case hit hua, `7` ne turant khud ko upar return kar diya. Yaani `2` ke left se answer aaya **`7`**.
- **Step 4: Right side of 2 processing (`2 -> 4`)**
- `2` ne apne right `4` par call lagayi.
- `4` khud humara target `q` hai! Base case hit hua, `4` ne khud ko upar return kar diya. Yaani `2` ke right se answer aaya **`4`**.
- **Step 5: Node `2` par decision**
- Node `2` ke paas Left se aaya `7` aur Right se aaya `4`.
- **Condition 1 Triggered!** Dono answers non-null hain. Iska matlab `2` hi inka **LCA** hai. `2` ne apna address upar return kar diya `5` ko.
- **Step 6: Node `5` par decision**
- `5` ke left se aaya tha `null` (step 2 se) aur right se aaya `2` (step 5 se).
- **Condition 2 Triggered!** Ek side null hai, toh jo valid hai use upar bhejo. `5` ne upar `3` ko return kiya **`2`**.

- **Step 7: Main Root 3 ke right side processing**
- 3 ne right side 1 par call lagayi thi, wahan na 7 hai na 4, toh wahan se return hokar aayega **null**.
- **Step 8: Final Decision at Root 3**
- Root 3 ke left se aaya 2 aur right se aaya null.
- Ek side valid hai, toh final answer ho gaya **2**.

---

4. C++ Code (Optimal Approach)

```
#include <iostream>

using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        // Base Case: Agar node null hai ya fir target node mil gaya hai
        if (root == nullptr || root == p || root == q) {
            return root;
        }

        // Left aur Right subtrees me targets ko dhoondho (Bottom-Up)
        TreeNode* leftAns = lowestCommonAncestor(root->left, p, q);
        TreeNode* rightAns = lowestCommonAncestor(root->right, p, q);

        // --- Post-order style decision making ---

        // Case 1: Agar dono side se valid answers mile hain, matlab yeh node khud
LCA hai
        if (leftAns != nullptr && rightAns != nullptr) {
            return root;
        }

        // Case 2: Agar left se kuch nahi mila, toh right se jo aaya wahi answer
hai
        if (leftAns == nullptr) {
            return rightAns;
        }

        // Case 3: Agar right se kuch nahi mila, toh left wala answer hai
        return leftAns;
    }
```

```
};
```

---

5. Complexity Breakdown

Time Complexity: $O(N)$

Hum pure tree ke har ek node ko maximum **ek hi baar** visit kar rahe hain standard DFS (Depth First Search) ki tarah. Kisi bhi node par koi loop nahi chal raha, sirf $O(1)$ ke null checks ho rahe hain. Isliye time complexity strictly linear hai.

Space Complexity: $O(H)$ / $O(N)$

- **Extra Space:** $O(1)$ kyunki humne rasta store karne ke liye koi alag se vector ya list nahi banayi.
- **Auxiliary Space:** Yeh recursion call stack ka space hai jo tree ki height (H) ke barabar hota hai. Worst case (skewed tree) me yeh $O(N)$ ho sakta hai, aur balanced tree me $O(\log N)$ hota hai.

Interview Tip: "Bina Vector Store Kiye Kaise Socha?"

Interviewer ko bolna: "*Sir, brute force me hum top-down soch rahe the jisme path store karna majboori tha. Optimal me mene **Post-Order traversal (Bottom-Up approach)** ka use kiya. Isme har child apne parent ko report karta hai ki niche target mila ya nahi. Jaise hi koi aisa parent milta hai jise left aur right dono se positive report milti hai, wahi point humara LCA ban jata hai.*"

Binary Search Tree (BST) me ek **given range `[low, high]`** ke beech ke saare elements ko sorted order me print karne ka problem kafi popular hai. Yeh problem pure tarike se BST ki core property par baithta hai, jahan hume poora tree bina wajah traverse nahi karna padta (hum irrelevant branches ko cut/prune kar dete hain).

Ise bilkul detail me, intuition aur dry run ke sath samajhte hain.

---

1. Core Intuition (BST Property ka Sahi Use)

BST ka niyam hume pata hai: `Left Subtree < Root < Right Subtree`. Humse kaha gaya hai ki elements **sorted order** me print hone chahiye. Kisi bhi BST ko agar sorted order me print karna ho, toh hum **Inorder Traversal (Left $\rightarrow$ Root $\rightarrow$ Right)** ka use karte hain.

Lekin standard Inorder me hum pure tree ke har ek node par jaate hain ($O(N)$). Range wale problem me hum thoda dimaag lagayenge:

- Agar current node ki value `low` se bhi **chhoti** hai, toh iska matlab iske left subtree me jaane ka koi fayda nahi hai (kyunki left wale toh isse bhi chhote honge). Hum seedhe **Right Subtree** me jayenge.
- Agar current node ki value `high` se **badi** hai, toh iska matlab iske right subtree me jaane ka koi fayda nahi hai (kyunki right wale toh isse bhi bade honge). Hum seedhe **Left Subtree** me jayenge.
- Agar current node ki value range ke **andar** hai (`low <= root->val <= high`), toh hume dono side check karna pad sakta hai aur is node ko print bhi karna hoga.

---

2. Step-by-Step Algorithm (Inorder Style)

Hum ek recursive function likhenge jo teen simple conditions check karega:

1. **Base Case:** Agar `root == nullptr`, toh chupchaap return ho jao.
2. **Left Call Decision:** Hum left subtree me tabhi jayenge agar current `root->val > low` ho. (Kyunki agar root pehle se hi `low` se chhota ya barabar hai, toh uske left me range ka koi element mil hi nahi sakta).
3. **Print Decision:** Agar current `root->val` range ke andar fall karta hai (`root->val >= low && root->val <= high`), toh use print kar do.
4. **Right Call Decision:** Hum right subtree me tabhi jayenge agar current `root->val < high` ho. (Kyunki agar root pehle se hi `high` se bada ya barabar hai, toh uske right me saare elements range se bahar honge).

*Note: Left call pehle lagane aur Right call baad me lagane se output hamesha automatic **Sorted Order** me aayega.*

---

3. Detailed Dry Run with an Example

Maan lete hain humare paas yeh BST hai:

```
      20
     /  \
    8    22
   / \
  4  12
```

Aur hume range di gayi hai: **`low = 10`** aur **`high = 22`**. *(Yani hume 10 se 22 ke beech ke elements sorted order me chahiye: `12, 20, 22`)*

- **Step 1: Main Root `20` par aaye.**
- Kya `20 > low (10)`? **Haan!** Toh Left Subtree (`8`) par jao.
- **Step 2: Node `8` par aaye.**
- Kya `8 > low (10)`? **Nahi!** (8 chhota hai, toh iske left `4` par jaane ka koi fayda nahi, skip `4`).
- Kya `8` range `[10, 22]` me hai? **Nahi.** (Print nahi hoga).
- Kya `8 < high (22)`? **Haan!** Toh Right Subtree (`12`) par jao.
- **Step 3: Node `12` par aaye.**
- Kya `12 > low (10)`? **Haan!** Left me call lagega, par `12->left` null hai, return.
- Kya `12` range `[10, 22]` me hai? **Haan!** → **Print: 12**
- Kya `12 < high (22)`? **Haan!** Right me call lagega, par `12->right` null hai, return.

- *Node 12 ka kaam khatam, back track to 20.*
- **Step 4: Wapas Root `20` par aaye (Left call complete ho chuki hai).**
- Kya `20` range `[10, 22]` me hai? **Haan!** → **Print: 20**
- Kya `20 < high (22)`? **Haan!** Toh Right Subtree (`22`) par jao.
- **Step 5: Node `22` par aaye.**
- Kya `22 > low (10)`? **Haan!** Left me call lagega, par null hai, return.
- Kya `22` range `[10, 22]` me hai? **Haan!** → **Print: 22**
- Kya `22 < high (22)`? **Nahi!** (22 already high ke barabar hai, toh iske right me jaane ka koi matlab nahi).

Final Output: `12 20 22` (Perfect sorted order!)

---

4. C++ Code Implementation

```cpp
#include <iostream>

using namespace std;

// BST Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void printRangeElements(TreeNode* root, int low, int high) {
        // Base Case
        if (root == nullptr) return;

        // Step 1: Left subtree me tabhi jao jab current node 'low' se bada ho
        if (root->val > low) {
            printRangeElements(root->left, low, high);
        }

        // Step 2: Agar current node range ke andar hai, toh print karo
        if (root->val >= low && root->val <= high) {
            cout << root->val << " ";
        }

        // Step 3: Right subtree me tabhi jao jab current node 'high' se chhota ho
        if (root->val < high) {
            printRangeElements(root->right, low, high);
```

```
            }
        }
    };

    // Helper function to insert nodes and build the tree
    TreeNode* insert(TreeNode* root, int val) {
        if (!root) return new TreeNode(val);
        if (val < root->val) root->left = insert(root->left, val);
        else root->right = insert(root->right, val);
        return root;
    }

    int main() {
        TreeNode* root = nullptr;
        root = insert(root, 20);
        root = insert(root, 8);
        root = insert(root, 22);
        root = insert(root, 4);
        root = insert(root, 12);

        Solution solver;
        int low = 10, high = 22;

        cout << "Elements in range [" << low << ", " << high << "] are: ";
        solver.printRangeElements(root, low, high); // Output: 12 20 22
        cout << endl;

        return 0;
    }
```

---

5. Complexity Analysis

Time Complexity:

- **Worst Case:** $O(N)$ Agar range bahut badi hai (jaise pure tree ke saare elements range me aa rahe hain), toh hume har ek node par jaana hi padega.
- **Average Case:** $O(H + K)$ Jahan H tree ki height hai aur K un nodes ki ginti hai jo range ke andar hain. BST property ki wajah se hum फालतू (irrelevant) branches ko pehle hi drop kar dete hain, isliye yeh standard traversal se kafi tez chalta hai.

Space Complexity: $O(H)$

Hum koi extra space use nahi kar rahe hain. Jo bhi space lag raha hai woh recursion stack ka hai, jo tree ki height (H) ke barabar hota hai. Balanced tree ke liye $O(\log N)$ aur skewed tree ke liye $O(N)$.

"Check if BST Contains Dead End" ek bahut hi dilchasp aur famous problem hai. Interviewer jab yeh question poochta hai, toh woh yeh dekhna chahta hai ki aap BST ke ranges aur properties ko kitne acche se samajhte hain.

Chalo isko bilkul aam bhasha me, core intuition aur detailed code ke sath samajhte hain.

---

1. Dead End Ka Matlab Kya Hai? (The Core Concept)

BST me **Dead End** us leaf node ko kehte hain jiske aage hum **koi bhi naya node insert nahi kar sakte**, kyunki uske left aur right subtree dono block ho chuke hote hain.

Hum jante hain ki BST me saare elements unique hote hain aur values hamesha positive integers (> 0) hoti hain (standard problem constraints ke hisab se).

Ek simple example se samjho: Agar aapke paas ek node hai jiski value hai `2`, aur uske tight constraints ki wajah se uske left me sirf `1` aa sakta hai aur right me sirf `3`. Agar tree me `1` aur `3` pehle se hi kahin insert ho chuke hain, toh `2` ke niche ab naya node banna impossible hai! Kyunki `1` aur `3` ke beech me koi dusra integer bacha hi nahi.

> **Golden Rule of Dead End:** Agar kisi node ki value `X` hai, aur uski valid range ka **Lower Bound `X - 1`** ban chuka hai aur **Upper Bound `X + 1`** ban chuka hai, toh woh node ek **Dead End** hai. *Special Case for 1:* Agar node ki value `1` hai, toh uske left me `0` nahi aa sakta (kyunki positive integers allowed hain). Agar `2` pehle se tree me hai, toh `1` bhi ek dead end ban jata hai kyunki uske lower side me koi space nahi hai aur upper side `2` ne block kar di.

---

2. Thought Process & Intuition (Kaise Sochein?)

Brute force me log pehle saare nodes ko ek hash set me daalte hain aur fir har leaf node ke liye `X-1` aur `X+1` check karte hain.

Lekin **Optimal Approach** me hum bina kisi extra memory (Set) ke, tree ko traverse karte-karte hi har node ki **Laxman Rekha (Range Boundaries)** track karte hain.

Hum top-down recursion chalayenge aur har node ke sath do cheezein pass karenge: `min_allowed` aur `max_allowed`.

- Shuruat me Root ke liye range hogi `[1, INT_MAX]`.
- Jab hum **Left** jayenge, toh upper bound restrict ho jayega: `[min_allowed, root->val - 1]`.
- Jab hum **Right** jayenge, toh lower bound restrict ho jayega: `[root->val + 1, max_allowed]`.

Jaise hi hum kisi node par pahunche aur humne dekha ki uska `min_allowed` aur `max_allowed` dono aapas me barabar ho gaye hain (`min_allowed == max_allowed`), iska matlab is node ke paas aage badhne ke liye koi integer bacha hi nahi hai. Yeh ek **Dead End** hai!

---

3. Detailed Dry Run with an Example

Maan lete hain humare paas yeh BST hai:

```
      8
     / \
    5   11
```

```
     / \
    2   7
   /
  1
```

Chalo check karte hain ki isme koi dead end hai ya nahi.

- **Root = 8:** Initial Range `[1, INT_MAX]`
- Left Call → Node `5`, Range ho gayi `[1, 8 - 1]` = `[1, 7]`
- **Node = 5:** Current Range `[1, 7]`
- Left Call → Node `2`, Range ho gayi `[1, 5 - 1]` = `[1, 4]`
- **Node = 2:** Current Range `[1, 4]`
- Left Call → Node `1`, Range ho gayi `[1, 2 - 1]` = `[1, 1]`
- **Node = 1:** Current Range `[1, 1]`
- Yahan dhyan se dekho! `min_allowed (1) == max_allowed (1)`.
- Iska matlab `1` ke aage koi space nahi bachi. Agar iske left me jayenge toh range `[1, 0]` banegi (invalid) aur right me `[2, 1]` (invalid).
- Mil gaya chor! Yeh node `1` ek **Dead End** hai. Hum turant `true` return kar denge.

---

4. C++ Code Implementation (Optimal Approach)

```cpp
#include <iostream>
#include <climits>

using namespace std;

// BST Node Structure
struct Node {
    int data;
    Node *left, *right;
    Node(int x) {
        data = x;
        left = right = nullptr;
    }
};

class Solution {
private:
    // Helper function jo har node ke liye valid range track karega
    bool checkDeadEnd(Node* root, int min_allowed, int max_allowed) {
        // Base Case: Agar null node hai, toh yahan dead end nahi ho sakta
```

```
        if (root == nullptr) return false;

        // Agar min aur max boundaries aapas me takra gayi hain,
        // toh iska matlab is node ke niche koi naya integer insert nahi ho sakta.
        if (min_allowed == max_allowed) {
            return true;
        }

        // Left subtree me range strictly restrict ho jayegi: [min, current_data - 1]
        // Right subtree me range strictly restrict ho jayegi: [current_data + 1, max]
        return checkDeadEnd(root->left, min_allowed, root->data - 1) ||
               checkDeadEnd(root->right, root->data + 1, max_allowed);
    }

public:
    bool isDeadEnd(Node* root) {
        // Problem ke mutabik saare elements positive integers (> 0) hain.
        // Isliye shuruat me min_allowed = 1 hoga aur max_allowed = INT_MAX.
        return checkDeadEnd(root, 1, INT_MAX);
    }
};

// Helper function to insert nodes in BST
Node* insert(Node* root, int data) {
    if (root == nullptr) return new Node(data);
    if (data < root->data) root->left = insert(root->left, data);
    else root->right = insert(root->right, data);
    return root;
}

int main() {
    Node* root = nullptr;
    root = insert(root, 8);
    root = insert(root, 5);
    root = insert(root, 11);
    root = insert(root, 2);
    root = insert(root, 7);
    root = insert(root, 1);

    Solution solver;
    if (solver.isDeadEnd(root)) {
        cout << "Yes, the BST contains a Dead End!" << endl; // Is case me Yes aayega
    } else {
        cout << "No Dead End found in the BST." << endl;
    }

    return 0;
}
```

5. Complexity Analysis

Time Complexity: $O(N)$

Hum pure tree ke har ek node ko strictly **sirf ek baar** visit kar rahe hain standard Preorder/DFS traversal ki tarah. Har node par hum sirf $O(1)$ ka constant comparison (`min == max`) kar rahe hain. Isliye time complexity strictly linear hai.

Space Complexity: $O(H)$

Humne koi extra data structure (jaise Hash Set ya Vector) use nahi kiya hai. Jo bhi space lag raha hai woh recursion call stack ka hai, jo tree ki height (H) ke barabar hota hai.

- Balanced BST ke liye: $O(\log N)$
- Skewed BST ke liye (worst case): $O(N)$

Yeh approach interview ke liye absolute best aur optimal hai kyuki yeh bina kisi extra storage ke single pass me result de deti hai!

Do (2) Binary Search Trees (BSTs) me **Common Nodes (yaani aise elements jo dono trees me present hain)** dhoondhne ka **Brute Force approach** kafi simple aur intuitive hai. Jab hume dono trees ke structures ya heights ke baare me kuch na pata ho, toh hum bilkul basic searching aur traversal logic ka use karte hain.

Is approach ko bilkul detail me, thought process aur dry run ke sath samajhte hain.

---

1. Brute Force Ka Thought Process (Kaise Sochein?)

Brute force tarike me hum dono BSTs ko bilkul alag-alag linear arrays ya structures ki tarah dekhte hain. Humare dimaag me sabse pehla khayal yeh aayega:

> "Kyun na main **Pehle Tree (BST 1)** ke har ek node par bari-bari se jaun... aur us node ki value ko **Doosre Tree (BST 2)** me jaakar dhoondhu (search karu)? Agar woh value BST 2 me mil jaati hai, toh iska matlab woh node dono me common hai!"

Hum BST ki property ka use kaise karenge?

Kyunki doosra tree ek **BST** hai, isliye jab hum Tree 1 ke kisi element ko Tree 2 me dhoondhne jayenge, toh hume poora Tree 2 scan karne ki zaroorat nahi hai. Hum BST ki standard **Binary Search property** use karenge:

- Agar target value current node se chhoti hai, toh left me dhoondho.
- Agar target value current node se badi hai, toh right me dhoondho.
- Kisi bhi element ko BST me search karne ka time $O(H_2)$ hota hai (jahan H_2 dusre tree ki height hai).

---

2. Detailed Algorithm Steps

Hum do functions ka use karenge:

1. `searchInBST(root2, value)`: Yeh function Tree 2 ke root aur ek target value ko lega. Yeh BST ke standard property se check karega ki value Tree 2 me hai ya nahi (returns `true` or `false`).

2. `findCommonNodes(root1, root2)`: Yeh humara main function hoga jo Tree 1 ka **Inorder Traversal (Left → Root → Right)** karega. Inorder traversal use karne ka ek fayda yeh hai ki jo common elements print honge, woh automatic **sorted order** me milenge.

Traversal Flow:

- Tree 1 ke Left subtree par jao.
- Tree 1 ka current node uthao → Use Tree 2 me search karo. Agar mil jaye, toh print/store kar lo.
- Tree 1 ke Right subtree par jao.

---

3. Detailed Dry Run with an Example

Maan lete hain humare paas yeh do BSTs hain:

BST 1:

```
      5
     / \
    1   10
   /     \
  0       12
```

BST 2:

```
     10
    /  \
   4    20
  / \
 1   7
```

Hum Tree 1 ka Inorder Traversal shuru karenge. Tree 1 ka sorted order sequence hai: `0, 1, 5, 10, 12`.

- **Step 1: Element = 0**
- BST 2 me `0` ko search karo:
- `10` par aaye. `0 < 10`, left me jao.
- `4` par aaye. `0 < 4`, left me jao.
- `1` par aaye. `0 < 1`, left me jao (null mila).
- Result: **Not Found**.
- **Step 2: Element = 1**

- BST 2 me `1` ko search karo:
- `10` par aaye. `1 < 10`, left jao.
- `4` par aaye. `1 < 4`, left jao.
- `1` par aaye. `1 == 1` → **Match Found!**
- Result: **Common Node = 1**.
- **Step 3: Element = 5**
- BST 2 me `5` ko search karo:
- `10` par aaye. `5 < 10`, left jao.
- `4` par aaye. `5 > 4`, right jao.
- `7` par aaye. `5 < 7`, left jao (null mila).
- Result: **Not Found**.
- **Step 4: Element = 10**
- BST 2 me `10` ko search karo:
- `10` par aaye. `10 == 10` → **Match Found!**
- Result: **Common Node = 10**.
- **Step 5: Element = 12**
- BST 2 me `12` ko search karo:
- `10` par aaye. `12 > 10`, right jao.
- `20` par aaye. `12 < 20`, left jao (null mila).
- Result: **Not Found**.

Final Common Nodes List: `1, 10`

---

4. C++ Code Implementation (Brute Force)

```cpp
#include <iostream>
#include <vector>

using namespace std;

// BST Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
```

```
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    // Helper function: BST 2 me kisi value ko search karne ke liye
    bool searchInBST(TreeNode* root, int target) {
        if (root == nullptr) return false;

        if (root->val == target) return true;

        // Agar target chhota hai toh left me dhoondho, nahi toh right me
        if (target < root->val) {
            return searchInBST(root->left, target);
        } else {
            return searchInBST(root->right, target);
        }
    }

public:
    // Main function: Tree 1 ka inorder traversal karte hue common nodes dhoondhna
    void findCommon(TreeNode* root1, TreeNode* root2, vector<int>& result) {
        if (root1 == nullptr) return;

        // 1. Left Subtree par jao
        findCommon(root1->left, root2, result);

        // 2. Current root ki value ko Tree 2 me search karo
        if (searchInBST(root2, root1->val)) {
            result.push_back(root1->val); // Agar mil gayi toh common hai
        }

        // 3. Right Subtree par jao
        findCommon(root1->right, root2, result);
    }
};

// Helper to insert nodes
TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

int main() {
    // Tree 1 construction
    TreeNode* root1 = nullptr;
    root1 = insert(root1, 5);
    root1 = insert(root1, 1);
    root1 = insert(root1, 10);
    root1 = insert(root1, 0);
    root1 = insert(root1, 12);
```

```
    // Tree 2 construction
    TreeNode* root2 = nullptr;
    root2 = insert(root2, 10);
    root2 = insert(root2, 4);
    root2 = insert(root2, 20);
    root2 = insert(root2, 1);
    root2 = insert(root2, 7);

    Solution solver;
    vector<int> commonNodes;
    solver.findCommon(root1, root2, commonNodes);

    cout << "Common nodes in both BSTs are: ";
    for (int val : commonNodes) {
        cout << val << " "; // Output: 1 10
    }
    cout << endl;

    return 0;
}
```

---

5. Complexity Analysis

Time Complexity:

- **Worst Case:** $O(N_1 \times N_2)$ Agar dono trees skewed trees hain (line jaise), toh Tree 1 ke har node (N_1) ke liye Tree 2 me search karne ka worst case time $O(N_2)$ ho jayega. Total time $O(N_1 \times N_2)$.
- **Average Case:** $O(N_1 \log N_2)$ Agar Tree 2 ek balanced BST hai, toh usme kisi bhi element ko search karne ki height complexity $\log N_2$ hoti hai. Tree 1 ke saare N_1 nodes ke liye yeh search chalega, isliye average time $O(N_1 \log N_2)$ ho jata hai.

Space Complexity: $O(H_1 + H_2)$

Hum koi extra temporary storage array/hash map use nahi kar rahe hain nodes ko store karne ke liye (sirf output list ko chhodkar). Jo space lag raha hai woh dono trees ke recursion call stacks ka hai:

- Tree 1 ke inorder traversal ka stack depth: $O(H_1)$
- Uske andar chalne wale Tree 2 ke search function ka stack depth: $O(H_2)$
- Total auxiliary space complexity: $O(H_1 + H_2)$.

Interview Note: Yeh Optimal Kyun Nahi Hai?

Bhale hi yeh approach space bacha rahi hai, par time complexity ke mamle me yeh thodi piche reh jaati hai. Interviewer aapse kahega: "*Kya tum ise strictly $O(N_1 + N_2)$ time me bina nested searching ke solve kar sakte ho?*" Uske do optimal tarike hote hain:

1. **Hash Map/Set approach:** Ek tree ko set me daal do aur dusre se match karo ($O(N_1 + N_2)$ time, $O(N_1)$ space).
2. **Two Pointer / Inorder Stack approach:** Dono trees ka simultaneous iterative inorder chalao standard merge sorted arrays ki tarah ($O(N_1 + N_2)$ time, $O(H_1 + H_2)$ space).

Do BSTs me common nodes dhoondhne ke **do (2) optimal tarike** hote hain, aur dono hi ise linear time $O(N1 + N2)$ me solve kar dete hain.

Dono approaches ke piche ka thought process aur unka implementation bilkul detail me samajhte hain.

Approach 1: Hash Set Method (Speed Optimal)

1. Kaise Socha? (The Intuition)

Brute force me dikkat yeh thi ki hum `BST 2` me baar-baar tree traversal karke search kar rahe the, jisme $\log N$ ya N ka time lag raha tha. Agar hume kisi element ko $O(1)$ time me dhoondhna ho, toh sabse best data structure hota hai **Hash Set**.

Hum kya karenge:

- `BST 1` ka poora traversal karke uske saare elements ko ek **Hash Set** me daal denge.
- Fir `BST 2` ko traverse karenge aur check karenge ki kya current node ki value Hash Set me pehle se maujood hai. Agar hai, toh woh common node hai!

2. C++ Code (Hash Set Method)

```cpp
#include <iostream>
#include <vector>
#include <unordered_set>

using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    // Helper function: BST 1 ke saare elements ko set me daalne ke liye
    void storeNodes(TreeNode* root, unordered_set<int>& s) {
        if (root == nullptr) return;
        s.insert(root->val);
        storeNodes(root->left, s);
        storeNodes(root->right, s);
    }

    // Helper function: BST 2 me common nodes dhoondhne ke liye
```

```
    // Inorder traversal use karne se result automatic sorted milega
    void checkCommon(TreeNode* root, unordered_set<int>& s, vector<int>& result) {
        if (root == nullptr) return;

        checkCommon(root->left, s, result);

        // Agar element set me mil jata hai toh woh common hai
        if (s.find(root->val) != s.end()) {
            result.push_back(root->val);
        }

        checkCommon(root->right, s, result);
    }

public:
    vector<int> findCommonNodes(TreeNode* root1, TreeNode* root2) {
        unordered_set<int> s;
        vector<int> result;

        storeNodes(root1, s);      // O(N1)
        checkCommon(root2, s, result); // O(N2)

        return result;
    }
};
```

3. Complexity

- **Time Complexity:** $O(N1 + N2)$ — Humne dono trees ko sirf ek-ek baar poora visit kiya. Set me search aur insert $O(1)$ average time leta hai.
- **Space Complexity:** $O(N1)$ — Pehle tree ke elements ko store karne ke liye Hash Set ka space lag raha hai.

Approach 2: Two-Pointer / Iterative Inorder Method (Space Optimal)

Agar interviewer aapse kahe: **"Mujhe $O(N1 + N2)$ time toh chahiye, par Hash Set ka use mat karo ($O(1)$ extra data structure space chahiye)"**, toh yeh approach kaam aati hai.

1. Kaise Socha? (The Intuition)

BST ki sabse badi khubi hoti hai ki uska **Inorder Traversal hamesha ek Sorted Array** deta hai.

Socho agar tumhare paas do sorted arrays hon, toh tum unme common elements kaise dhoondhte ho? **Two-Pointer Approach** se! Tum dono arrays ke shuruat me ek-ek pointer rakhte ho, jo element chhota hota hai uske pointer ko aage badha dete ho, aur agar match mil jaye toh use common maan lete ho.

Lekin hume tree ko array me convert nahi karna (kyunki usme extra space lag jayega). Isliye hum **Iterative Inorder Traversal using Stack** ka use karke dono BSTs ko *ek sath (simultaneously)* traverse karenge.

2. Step-by-Step Mechanism

1. Do stacks banayenge: `s1` (BST 1 ke liye) aur `s2` (BST 2 ke liye).
2. Dono trees ke sabse leftmost nodes tak travel karke stack me push karenge (jaise normal iterative inorder me karte hain).
3. Ab dono stacks ke top elements ko compare karenge:

- **Agar `top1->val < top2->val`:** Iska matlab BST 1 ka element chhota hai. Hum `s1` se pop karenge aur BST 1 me us node ke right subtree par move kar jayenge.
- **Agar `top1->val > top2->val`:** Iska matlab BST 2 ka element chhota hai. Hum `s2` se pop karenge aur BST 2 me us node ke right subtree par move kar jayenge.
- **Agar `top1->val == top2->val`:** Hume **Common Node** mil gaya! Hum dono ko pop karenge, print/store karenge, aur dono ke right subtrees par move kar jayenge.

3. C++ Code (Space Optimal - Two Stacks)

```cpp
#include <iostream>
#include <vector>
#include <stack>

using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    vector<int> findCommonNodesOptimal(TreeNode* root1, TreeNode* root2) {
        vector<int> result;
        stack<TreeNode*> s1, s2;

        TreeNode* curr1 = root1;
        TreeNode* curr2 = root2;

        while (true) {
            // BST 1 ke extreme left tak jao
            while (curr1 != nullptr) {
                s1.push(curr1);
                curr1 = curr1->left;
            }

            // BST 2 ke extreme left tak jao
            while (curr2 != nullptr) {
                s2.push(curr2);
                curr2 = curr2->left;
            }
```

```
            // Agar koi bhi stack khali ho gaya, toh aage common elements nahi mil
sakte
            if (s1.empty() || s2.empty()) break;

            TreeNode* top1 = s1.top();
            TreeNode* top2 = s2.top();

            // Case 1: Match mil gaya
            if (top1->val == top2->val) {
                result.push_back(top1->val);
                s1.pop();
                s2.pop();
                curr1 = top1->right; // Right subtree me move karo
                curr2 = top2->right;
            }
            // Case 2: BST 1 ka element chhota hai, toh uske pointer ko badhao
            else if (top1->val < top2->val) {
                s1.pop();
                curr1 = top1->right;
                curr2 = nullptr; // BST 2 ko vahi hold karke rakho
            }
            // Case 3: BST 2 ka element chhota hai
            else {
                s2.pop();
                curr2 = top2->right;
                curr1 = nullptr; // BST 1 ko vahi hold karke rakho
            }
        }
        return result;
    }
};
```

4. Complexity

- **Time Complexity:** $O(N1 + N2)$ — Hum dono trees ke har element ko maximum ek hi baar stack me daal aur nikal rahe hain. Linear time optimization achieve ho gayi.
- **Space Complexity:** $O(H1 + H2)$ — Hum koi extra element storage use nahi kar rahe hain. Jo space lag raha hai woh sirf stacks ka hai, jo balanced tree ke liye sirf $O(\log N1 + \log N2)$ hota hai. Yeh Hash Set approach ($O(N)$ space) se kafi zyada efficient hai memory ke mamle me.